



UNIVERSIDAD DE MONTERREY

Practica 1

Microcontroladores

Alejandro Reyes

Nombre:

Mat:

Nombre: Ivan Alejandro Navarro Alonso

Mat: 617640

Nombre: David Cabrera

Mat:649032

“Damos nuestra palabra de que hemos hecho esta actividad con integridad académica”

Monterrey, N. L. a 11 de junio del 2026

Introducción

Los microcontroladores PIC permiten controlar dispositivos digitales sencillos, como LEDs, a través del manejo de sus puertos de entrada y salida. En esta práctica se desarrollaron dos programas en lenguaje C, compilados con XC8, que controlan el Puerto D de un microcontrolador PIC para generar dos patrones de iluminación distintos: un parpadeo simultáneo de los ocho LEDs y una secuencia de barrido tipo “Knight Rider”, en la que un solo LED se desplaza de un extremo del puerto al otro.

El objetivo es comprender cómo la configuración de los bits de configuración (fuses), la dirección de los puertos y los retardos por software determinan el comportamiento final del circuito.

Marco teórico

Registros TRISx y PORTx:

Cada puerto del microcontrolador tiene asociados dos registros: TRISx, que define si cada pin es entrada (1) o salida (0), y PORTx, que contiene el valor lógico escrito o leído en dichos pines. Para controlar LEDs, los pines deben configurarse como salida antes de poder encenderlos o apagarlos.

Bits de configuración y oscilador:

Los “fuses” definen el comportamiento del hardware antes de ejecutar el programa: tipo de oscilador (FOSC), Watchdog Timer (WDTE), temporizador de encendido (PWRT), reset por bajo voltaje (BOREN), programación de bajo voltaje (LVP) y protección de memoria (CP, CPD, WRT). El parámetro FOSC debe coincidir con el cristal físico utilizado, ya que de esto depende la precisión de los tiempos generados por software.

Retardos por software:

La función `__delay_ms()` genera una pausa aproximada en milisegundos, calculada internamente a partir de la macro `_XTAL_FREQ`. Si esta macro no coincide con la frecuencia real del oscilador configurado, los tiempos de retardo reales serán distintos a los programados.

Desarrollo

Código 1 — Parpadeo del Puerto D:

El primer programa configura los ocho pines del Puerto D como salida y, dentro de un ciclo infinito, enciende los ocho LEDs de forma simultánea durante 500 ms y después los apaga durante otros 500 ms, repitiendo este ciclo indefinidamente. El oscilador se configura en modo XT, adecuado para cristales de hasta aproximadamente 4 MHz, mientras que el Watchdog Timer, la programación de bajo voltaje y la protección de memoria permanecen desactivados; el reset por bajo voltaje sí está habilitado.

Se identificó una inconsistencia en este programa: la macro de frecuencia del oscilador se declaró en 8 MHz, mientras que el oscilador configurado (XT) opera en un rango de hasta 4 MHz. Esta diferencia provoca que los retardos calculados por software no coincidan con el tiempo real en hardware, por lo que el parpadeo observado en el circuito físico sería más lento de lo esperado. Para corregirlo, la frecuencia declarada debería ajustarse a 4 MHz, de acuerdo con el oscilador realmente utilizado.

Código 2 — Secuencia tipo Knight Rider:

El segundo programa también configura el Puerto D como salida, pero en lugar de encender todos los LEDs a la vez, desplaza un único LED encendido a lo largo del puerto. Primero se recorre de izquierda a derecha, comenzando en el primer pin y avanzando uno a uno hasta el último, y después se recorre de regreso en sentido contrario, generando un efecto de “ojo de auto” o rebote. El oscilador se configura en modo HS, y la frecuencia declarada (8 MHz) sí es coherente con este modo, por lo que en este caso los retardos de 500 ms entre cada paso corresponden al tiempo real en hardware.

El desplazamiento del LED se logra mediante una variable que actúa como máscara de bits, la cual se recorre con operaciones de desplazamiento: hacia la izquierda en el trayecto de ida y hacia la derecha en el trayecto de regreso. Esto permite generar el patrón de movimiento sin necesidad de almacenar una tabla de valores, aprovechando el desbordamiento natural de una variable de 8 bits para delimitar cada recorrido.

Comparación entre ambos códigos:

Aspecto	Código 1	Código 2
Oscilador (FOSC)	XT	HS
Frecuencia declarada	8 MHz	8 MHz (coherente)
Control de LEDs	Escritura directa del byte completo	Desplazamiento de un bit a la vez
Patrón visual	Los ocho LEDs encienden y apagan juntos	Un LED se desplaza de un extremo a otro

Conclusión

El desarrollo de esta práctica permitió comprobar que el control de LEDs mediante un microcontrolador PIC depende de tres elementos fundamentales: la configuración correcta de los bits de configuración del oscilador, la declaración apropiada de la dirección de los puertos mediante los registros TRISx, y la coherencia entre la frecuencia real del cristal y la macro utilizada para calcular los retardos por software. El primer programa, aunque sencillo, evidenció la importancia de mantener consistencia entre el oscilador configurado y la frecuencia declarada, ya que un descuido en este punto altera directamente los tiempos del sistema. El segundo programa, por su parte, mostró cómo el uso de operadores de desplazamiento de bits permite generar patrones de iluminación más elaborados de manera eficiente. En conjunto, ambos ejercicios reforzaron los conceptos básicos de configuración y manejo de puertos de entrada/salida en microcontroladores PIC.